SLIDE 1 • 00:00-01:00

Same AI. Different Results.

The title of this talk is Same AI, Different Results. I like it because it sounds almost unfair. We all have access to strong models now. We all have chat windows. We all have tools that can write, summarize, code, and plan. So why do the outcomes differ so much? Why does one team get something useful while another gets polished nonsense? My argument is simple. The difference is not mainly the model. The difference is whether the AI can work with the memory of the organization. In the age of AI, intelligence is becoming common. Context is not. Judgment is not. Shared memory is not. That is where the real difference starts.

SLIDE 2 • 01:00-02:00

The Familiar Scene

Let me start with a scene most of us know. Two people ask almost the same thing. Maybe it is a teacher asking for a lesson plan. Maybe it is a manager asking for a brief. Maybe it is a developer asking for an implementation plan. Both answers sound good. Both are fluent. But one fits the local reality and the other does not. One uses the right language. The other ignores a rule, misses an exception, or suggests something the team already rejected. The prompt was similar. The model was similar. The hidden context was not. That is the basic puzzle behind this talk.

Why Better Prompting Is Not Enough

When this happens, we often blame the prompt. We say we should have asked better. Sometimes that is true. But after a while that becomes a strange way to run a company. It means every person has to carry the organization in their own head and paste it into the chat again and again. Please remember our standards. Please remember our customer promises. Please remember why we built it this way. Please remember what legal said last month. That is not a system. That is humans acting as memory sticks for machines. The more agents we use, the more expensive that becomes. The memory needs a better home.

AI Is A Shock. Coherence Is The Question.

This matters because AI is not a small feature. It is a general-purpose technology. It changes many kinds of work at once. Some tasks get automated. Some get amplified. Some are changed indirectly because expectations change. That is why the jobs debate can be a bit too narrow. The question for this room is not only what AI will replace. The better question is this: when the tools multiply, what keeps the work coherent? What keeps a company, a public institution, or a professional practice from becoming a pile of disconnected outputs? That is where knowledge management becomes strategic again.

What Gets Lost

Most knowledge work is not only about outputs. The visible part is the document, the policy, the slide deck, the code, the answer to the customer. Underneath that sits judgment. Why did we choose this path? What risk were we trying to avoid? Why do we use this wording and not that one? Which exception matters in practice? An AI connected only to files can often see the output. It cannot automatically see the reasoning that produced it. That is why generic AI can sound smart and still be locally wrong. It sees the surface of the work, not the logic underneath it.

Two Kinds Of Context

A simple way to say this is that there are two kinds of context. First, there is what is true now. The current numbers. The current file. The current ticket. The current state of the world. That matters. But there is also why it became true. Why the team chose this path. What failed before. What constraint mattered. What exception keeps coming back. I like the shorthand: data context makes agents accurate. Decision context makes agents institutional. Usable is strongest in the second half of that sentence. It helps preserve the part organizations usually lose.

What Usable Is

So what is Usable really? Not mainly a knowledge base. Not a folder of PDFs. Not a wiki nobody updates. Not a frozen copy of every database. The best short description I found in our own material is this: Usable is organizational memory for AI agents. More precisely, governed organizational memory. A place where important decisions, standards, precedents, patterns, and lessons become available to people and agents at the moment work is happening. The point is not to make storage look clever. The point is to make real work less forgetful.

What Usable Is Not

There is another important part of the current Usable story. We should not pretend that all truth should live inside one memory system. That would create stale confidence very quickly. Live facts should often stay in the live systems where they belong. The current customer status. The current inventory. The current operational state. Usable is strongest around durable learning and decision context. Store the rules, the reasons, the standards, the lessons, the map of where truth lives, and the permissions around it. Then let the agent fetch volatile facts live. That is a much cleaner product story.

The Loop That Compounds

This is the whole idea in one picture. Without a memory layer, the loop is simple and wasteful: ask, get answer, forget. The next person starts again. The next agent starts again. With a memory layer, the loop changes. Read before acting. Act. Capture what mattered. Govern it. Reuse it next time. That sounds small, but it changes everything. It moves AI from disposable help to compounding capability. It also reflects the product pattern we keep seeing: the agent is grounded in a workspace, it can reach the right local tools, and when the work is done it leaves something useful behind.

Proof 1: TAKS

The TAKS case is a very good public example of this. The assistant is called Eydei, which sounds like AI in Faroese, and TAKS even gave the wider cast names like Tollakur and Skatta Ro. It is friendly on the surface, but the real point is deeper. Tax is high-trust work. People ask about deadlines, customs, deductions, and consequences. A generic bot is not enough. The assistant has to be grounded in official material. It has to be careful with sources. It has to improve from what happens in the real workflow. That is why this story matters. The value is not the chat face. The value is the governed memory behind it.

Proof 2: Julius

Yesterday at Reinsarid, we looked at this from several angles. Julius gave the software-development version. His point was not just that AI makes developers faster. His point was that speed creates a new process problem. Three good developers can ask the same AI for the same feature and get three reasonable but different implementations. None are obviously crazy. They just do not fit together. Then the team pays the alignment tax at review time. Julius showed a better loop: start from a PRD, split into aligned tasks, enhance the plan with standards and prior learning before code is written, then write the lesson back after the commit. That moves alignment from review time to plan time.

The Pattern Repeats

What struck me yesterday was not that we had several talks. It was that the same missing layer kept showing up in very different domains. In Jaspur's strategy work, the project memory became part of the deliverable. The value was not only the final document, but the reasoning trail behind it. In Johann's game-development work, the breakthrough came when the project stopped living only in the latest prompt. The agent got better when it could return to a workspace with history. Different domain. Same pattern. The tool gets more useful when it can stand on accumulated local context instead of improvising from scratch every time.

More Than One Interface

Brian and Ajit push the same idea further. Brian's operators showed that the important thing is not the surface. Slack, IDE, browser, terminal, phone: those are interfaces. The durable part is the memory layer underneath, plus the harness around it. Ajit then gave the conceptual language for the frontier. If we want agents to improve, we need more than retrieval. We need traces of how decisions were made and what happened next. That is when memory starts becoming precedent. And that is when knowledge management begins to feel modern again instead of old-fashioned.

What A Decision Trace Is

A decision trace is easier to explain than it sounds. It says: what did we observe, what constraint mattered, what tradeoff did we make, what action did we take, and what happened afterwards? That is different from a log. A log says what happened. A decision trace tries to preserve why it happened. This is one of the deepest points in the current Usable story. Organizations already store lots of output. Much fewer store chains of judgment that an agent can actually reuse. Once you preserve that chain, memory becomes more valuable than search alone.

Precedent And Boundaries

Ajit's context-graph framing is useful because it points beyond document retrieval. The goal is not just to find similar text. The goal is to understand relationships between decisions, exceptions, actions, and outcomes. But there is another deep point here. One big company brain is not the answer. The hard part is governing the overlap. The sales assistant should not see board material. The engineering agent should not get every private HR note. Good organizational memory is not only rich. It is well-bounded. That is why governed memory matters so much.

Every Profession Has This

This is not a software-only story. Every knowledge profession has local judgment that generic AI cannot guess. A teacher has examples that worked with a certain group. A lawyer has clause history and negotiation rationale. An engineer has incidents, standards, and scars. A manager has tradeoffs, promises, and stakeholder history. A founder has bets, signals, and lessons learned the hard way. The forms change, but the pattern does not. Good work depends on more than facts. It depends on remembered judgment in a specific context.

Personal Memory, Team Memory

There is also a personal version and a team version. Your own best examples, checklists, and explanations matter. So do the standards that live only in the heads of experienced teammates. The opportunity is to make that knowledge reusable without pretending everyone should see everything. This is why I keep coming back to the word governed. The future is not one giant pile of memory. It is shared memory where it helps, local memory where it belongs, and clear boundaries in between. That is a more realistic vision of AI at work.

Monday Morning, Part One

So what should someone in this room do on Monday morning? First, pick one repeated workflow. Not the whole organization. One workflow. Second, capture the decisions and standards that shape good work in that workflow. What do experienced people know that newcomers or agents do not? Third, connect the agent to that memory before it acts. Do not wait until the review stage to add context. Bring the context earlier, into the plan and the first draft. That alone changes the quality of what comes back.

Monday Morning, Part Two

Then do two more things. Let the agent reach the right local tools instead of guessing around them. And make the end of the task include a memory update. What changed? What was learned? What should the next person or next agent know? That is enough to start. You do not need a giant transformation program. You need one loop that stops throwing away learning. If you do that well, the benefit is not only speed. It is calm. Fewer repeated mistakes. Less re-explaining. Better first drafts. Better second runs.

Close

I want to end where I began. Same AI. Different Results. In the age of AI, many people can buy access to intelligence. That means advantage moves somewhere else. It moves to owned memory. It moves to judgment made durable. It moves to whether your agents can work with the reality of your world instead of only the patterns of the internet. That is the story I think Usable is really telling now. Not just a knowledge base. Not just search. Governed organizational memory for AI agents. Build that layer well and the same model starts giving you very different outcomes. Thank you.